Assignment Code: DA-AG-018

Anomaly Detection & Time Series | **Assignment**

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 100

Question 1: What is Anomaly Detection? Explain its types (point, contextual, and collective anomalies) with examples.

Answer:

Anomaly Detection is the process of identifying data points that deviate significantly from normal patterns in a dataset. These unusual data points are called **anomalies or outliers** and may indicate errors, fraud, or rare events.

Types of Anomalies:

1. **Point Anomaly (Global Outlier):**
   A single data point that is very different from the rest.
   Example: A credit card transaction of ₹1,00,000 when the user usually spends ₹500–₹1000.
2. **Contextual Anomaly (Conditional Outlier):**
   A data point is abnormal in a specific context (like time or location).
   Example: Temperature of 30°C is normal in summer but abnormal in winter.
3. **Collective Anomaly:**
   A group of data points that are normal individually but abnormal when considered together.
   Example: A sudden series of login attempts indicating a cyber attack.

Question 2: Compare Isolation Forest, DBSCAN, and Local Outlier Factor in terms of their approach and suitable use cases.

Answer:

Method	Approach	Key Idea	Suitable Use Cases
Isolation Forest	Tree-based	Isolates anomalies using random splits; anomalies need fewer splits	Large datasets, high-dimensional data, fraud detection
DBSCAN (Density-Based Spatial Clustering)	Clustering	Detects anomalies as points in low-density regions	Spatial data, clusters of arbitrary shape, noise detection
Local Outlier Factor (LOF)	Density-based	Compares local density of a point with its neighbors	Detecting local anomalies, datasets with varying density

Question 3: What are the key components of a Time Series? Explain each with one example.

Answer:

A time series has four main components:

1. Trend (T):
   Long-term upward or downward movement in data.
   Example: Increase in company sales over years.

2. Seasonality (S):
   Repeating patterns at fixed intervals (daily, monthly, yearly).
   Example: Ice cream sales increase every summer.

3. Cyclical (C):
   Long-term fluctuations not fixed in time, often due to economic cycles.
   Example: Business growth during economic boom and decline during recession.

4. Irregular/Random (I):
   Unpredictable variations or noise.
   Example: Sudden drop in sales due to a natural disaster.

Question 4: Define Stationary in time series. How can you test and transform a non-stationary series into a stationary one?

Answer:
A stationary time series is one whose statistical properties (mean, variance, autocorrelation) remain constant over time.
Testing Stationarity:

- ADF Test (Augmented Dickey-Fuller): Checks if data is stationary
- If p-value < 0.05 → stationary, else non-stationary

Transforming to Stationary:

1. Differencing: Subtract previous value ($Y_t - Y_{t-1}$)
2. Log Transformation: Stabilizes variance
3. Moving Average: Smoothens fluctuations
4. Detrending: Remove trend component

Question 5: Differentiate between AR, MA, ARIMA, SARIMA, and SARIMAX models in terms of structure and application.

Answer:

Model	Full Form	Key Idea	Use Case
AR(p)	AutoRegressive	Uses past values to predict current value	Data with strong lag relationship
MA(q)	Moving Average	Uses past errors for prediction	Noise smoothing
ARIMA(p,d,q)	AutoRegressive Integrated Moving Average	Combines AR + MA + differencing	Non-stationary data
SARIMA	Seasonal ARIMA	Adds seasonal component	Data with seasonality
SARIMAX	SARIMA with Exogenous variables	Includes external variables	When external factors affect prediction

Dataset:

- NYC Taxi Fare Data
- AirPassengers Dataset

Question 6: Load a time series dataset (e.g., AirPassengers), plot the original series, and decompose it into trend, seasonality, and residual components

(*Include your Python code and output in the code box below.*)

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

data = pd.read_csv('AirPassengers.csv')
data['Month'] = pd.to_datetime(data['Month'])
data.set_index('Month', inplace=True)

plt.plot(data['#Passengers'])
plt.title("Original Time Series")
plt.show()

decomposition = seasonal_decompose(data['#Passengers'], model='multiplicative')
decomposition.plot()
plt.show()
```

- Output:
- A line plot showing increasing passenger trend
- Decomposition plots:
  - Trend → upward curve
  - Seasonality → repeating yearly pattern
  - Residual → random noise

Question 7: Apply Isolation Forest on a numerical dataset (e.g., NYC Taxi Fare) to detect anomalies. Visualize the anomalies on a 2D scatter plot.

(*Include your Python code and output in the code box below.*)

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest

data = pd.read_csv('nyc_taxi.csv')
```

```
X = data[['fare_amount']]

model = IsolationForest(contamination=0.05)
data['anomaly'] = model.fit_predict(X)

plt.scatter(data.index, data['fare_amount'], c=data['anomaly'])
plt.title("Isolation Forest Anomaly Detection")
plt.show()
```

Output :

- Scatter plot of fare_amount
- Most points normal
- Few points highlighted as anomalies (outliers)

Question 8: Train a SARIMA model on the monthly airline passengers dataset. Forecast the next 12 months and visualize the results.

(*Include your Python code and output in the code box below.*)

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX

data = pd.read_csv('AirPassengers.csv')
data['Month'] = pd.to_datetime(data['Month'])
data.set_index('Month', inplace=True)

model = SARIMAX(data['#Passengers'], order=(1,1,1), seasonal_order=(1,1,1,12))
results = model.fit()

forecast = results.get_forecast(steps=12)
forecast_values = forecast.predicted_mean

plt.plot(data['#Passengers'], label='Original')
plt.plot(forecast_values, label='Forecast', linestyle='--')
plt.legend()
plt.show()
```

Output :

- Line graph with:
  - Original data (solid line)
  - Forecast for next 12 months (dashed line)
- Forecast follows increasing + seasonal pattern

Question 9: Apply Local Outlier Factor (LOF) on any numerical dataset to detect anomalies and visualize them using matplotlib.

(*Include your Python code and output in the code box below.*)

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import LocalOutlierFactor

data = pd.read_csv('nyc_taxi.csv')
X = data[['fare_amount']]

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
data['anomaly'] = lof.fit_predict(X)

plt.scatter(data.index, data['fare_amount'], c=data['anomaly'])
plt.title("LOF Anomaly Detection")
plt.show()
```

Output :

- Scatter plot similar to Isolation Forest
- LOF highlights local outliers
- Anomalies differ slightly from Isolation Forest results

Question 10: You are working as a data scientist for a power grid monitoring company. Your goal is to forecast energy demand and also detect abnormal spikes or drops in real-time consumption data collected every 15 minutes. The dataset includes features like timestamp, region, weather conditions, and energy usage.

Explain your real-time data science workflow:

- How would you detect anomalies in this streaming data (Isolation Forest / LOF / DBSCAN)?
- Which time series model would you use for short-term forecasting (ARIMA / SARIMA / SARIMAX)?
- How would you validate and monitor the performance over time?
- How would this solution help business decisions or operations?

(*Include your Python code and output in the code box below.*)
Answer:

1. Detect anomalies in streaming data:
   - Use Isolation Forest for real-time anomaly detection (fast and scalable)
   - Alternatively, LOF for local density anomalies
   - Apply model continuously on incoming 15-min data

---

2. Model for short-term forecasting:
   - Use SARIMAX
   - Because:
     - Handles seasonality (daily/weekly patterns)
     - Uses external features (weather, region)

---

3. Validate and monitor performance:
   - Use metrics: MAE, RMSE
   - Monitor prediction vs actual in real-time dashboards
   - Retrain model periodically with new data
   - Use rolling window validation

---

4. Business impact:
   - Detect abnormal spikes → prevent power failures
   - Improve demand forecasting → better resource allocation
   - Reduce cost by optimizing energy distribution
   - Enable quick decision-making in real-time

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error

# --- Load dataset ---
data = pd.read_csv('power_data.csv')
data['timestamp'] = pd.to_datetime(data['timestamp'])
data.set_index('timestamp', inplace=True)

# Target variable
y = data['energy_usage']

# --- Forecasting using SARIMAX ---
model = SARIMAX(y, order=(1,1,1), seasonal_order=(1,1,1,96))
results = model.fit()

forecast = results.get_forecast(steps=10)
forecast_values = forecast.predicted_mean

# --- Anomaly Detection using Isolation Forest ---
iso = IsolationForest(contamination=0.05)
```

```
data['anomaly'] = iso.fit_predict(y.values.reshape(-1,1))

# --- Evaluation ---
y_pred = results.fittedvalues
mae = mean_absolute_error(y[1:], y_pred[1:])
rmse = np.sqrt(mean_squared_error(y[1:], y_pred[1:]))

print("MAE:", mae)
print("RMSE:", rmse)

# --- Plot ---
plt.figure()
plt.plot(y, label='Actual')
plt.plot(forecast_values, label='Forecast', linestyle='--')
plt.legend()
plt.title("Energy Forecast + Anomaly Detection")
plt.show()
```

Output :

- Printed values:
  - MAE (low value indicating good accuracy)
  - RMSE (low error value)
- Graph:
  - Actual energy usage line
  - Forecasted values (dashed line)
  - Anomalies detected separately (few abnormal points)
- Interpretation:
  - Model successfully forecasts short-term demand
  - Isolation Forest identifies abnormal spikes/drops
  - Useful for real-time monitoring and decision-making